Manual de Implementación Técnica y UX de Scrollytelling de Alto Rendimiento

La construcción de narrativas digitales contemporáneas ha trascendido la mera disposición estática de información para convertirse en una disciplina de ingeniería que fusiona la psicología cognitiva, el arte técnico y la optimización extrema del rendimiento frontend. El scrollytelling, o narrativa impulsada por el desplazamiento, representa la culminación de este esfuerzo, permitiendo que el usuario no sea solo un consumidor de contenido, sino el motor cinético que activa la progresión de una historia.[1] El presente manual tiene como objetivo instruir a un Asistente de IA Senior Frontend & Design en la creación de experiencias verticales inmersivas que cumplan con los más altos estándares de fidelidad visual, accesibilidad universal y eficiencia computacional, basándose en patrones de diseño validados por instituciones como el Nielsen Norman Group (NN/g) y bibliotecas de vanguardia como GSAP y Framer Motion.[2]

1. Arquitectura de Sistemas Narrativos y Selección Tecnológica

La elección de la pila tecnológica es el cimiento sobre el cual se construye la experiencia del usuario. En el desarrollo de scrollytelling, la arquitectura debe decidirse en función del equilibrio entre el control creativo y la sobrecarga del hilo principal del navegador. La ingeniería moderna ofrece tres caminos principales: las APIs nativas de CSS para un rendimiento "fuera del hilo" (off-main-thread), GSAP para una secuenciación compleja y robusta, y Framer Motion para una integración profunda en ecosistemas React.[4]

Comparativa de Ecosistemas de Animación

Para un desarrollador Senior, entender las compensaciones entre estas herramientas es vital para la escalabilidad del proyecto. Mientras que el CSS nativo ofrece el mejor rendimiento bruto al delegar las animaciones al compositor de la GPU, carece de la lógica de control temporal y de los ganchos de eventos que poseen las librerías de JavaScript.[5]

Criterio de Evaluación	CSS Scroll-driven API	GSAP (GreenSock)	Framer Motion
Rendimiento de Renderizado	Máximo (GPU-accelerated, off-main-thread).	Muy Alto (Optimizado para manipulación de	Alto (Optimizado mediante MotionValues).

		propiedades).	
Control de Secuenciación	Limitado a keyframes básicos.	Máximo (Timelines, Staggers, Callbacks).	Alto (Declarativo, Layout Animations).
Soporte de Navegadores	Emergente (Safari 26+, Chrome 115+).[4]	Universal (IE11+ a través de polyfills).	Moderno (React 18+).
Accesibilidad Nativa	Integración directa con Media Queries.	Requiere manejo manual de contextos.	Sólido a través de props dedicadas.
Peso del Bundle	0 KB (Nativo).	~25-60 KB (Dependiendo de plugins).	~30 KB.

La tendencia para 2025 es la hibridación: utilizar CSS nativo para animaciones de visibilidad y progreso simples (como barras de lectura o efectos de opacidad) y reservar GSAP para las interacciones críticas donde el "Video Scrubbing" o la sincronización multi-capa es esencial para la narrativa.[9]

El Cambio de Paradigma: Del Tiempo al Espacio

Tradicionalmente, las animaciones se han definido en una línea de tiempo basada en milisegundos. El scrollytelling rompe este esquema al mapear la duración de la animación a la distancia recorrida en píxeles. En términos de arquitectura, esto significa que la función de tiempo $f(t)$ se reemplaza por una función de desplazamiento $f(y)$, donde y es la posición vertical del scroll.[4] La ingeniería avanzada debe tratar el scroll como una variable continua de entrada que el navegador debe muestrear a la mayor frecuencia posible para evitar el "jank" o tartamudeo visual, especialmente en dispositivos con pantallas de alta tasa de refresco.[11]

2. Fundamentos de UX y Psicología del Desplazamiento

La estrategia narrativa debe estar informada por la fisiología de la atención humana. Según estudios de Microsoft y Statista, el usuario promedio de 2025 opera con una capacidad de atención de aproximadamente 8 segundos antes de decidir si abandona un sitio.[12] El scrollytelling actúa como un antídoto a esta fragmentación al transformar la lectura pasiva en

un compromiso activo.[1]

Atención del Usuario y Patrones de Lectura

El Nielsen Norman Group ha documentado extensamente el patrón de lectura en "F", donde los usuarios escanean horizontalmente la parte superior y luego verticalmente el margen izquierdo.[1] El scrollytelling exitoso, como los famosos reportajes del *New York Times*, interrumpe este patrón mediante "puntos de anclaje visual" que fuerzan al ojo a detenerse y procesar información multimedia que se revela progresivamente.[14]

La investigación de Umeå University en 2023 confirmó que los usuarios se sienten más involucrados emocionalmente cuando el contenido se entrega a través de narrativas interactivas impulsadas por el desplazamiento.[2] Esta inmersión emocional se traduce en métricas tangibles: el tiempo de permanencia puede aumentar hasta un 85% y las tasas de clics hacia objetivos secundarios en un 47%.[1] Sin embargo, la efectividad depende de no abrumar al usuario con "ruido visual"; cada animación debe tener una justificación semántica.[12]

La Zona de Confort del Scroll y el Pacing

El *pacing* o ritmo es la velocidad a la que la historia progresa en relación con el esfuerzo físico del usuario. Una "Zona de Confort" óptima evita la fatiga del pulgar (en móviles) y la desorientación mental. Se ha identificado que los usuarios tienden a abandonar sitios si se les obliga a desplazarse demasiado para obtener poca recompensa visual.[1]

1. **Regla de Proporción de Avance:** Para visualizaciones de datos densas, el avance de la animación debe ser lento y granular (pacing bajo). Para transiciones atmosféricas, el avance puede ser más dinámico (pacing alto).[17]
2. **Fatiga de Interacción:** El exceso de disparadores de scroll puede agotar cognitivamente al usuario. El diseño debe permitir "momentos de respiración" con contenido estático o movimientos ambientales sutiles entre secuencias de alta intensidad.[12]

"Sticky Positioning" vs. "Parallax": Criterios de Aplicación

La decisión entre fijar un elemento o moverlo a diferente velocidad no es meramente estética, sino funcional.

- **Sticky Positioning (Posicionamiento Adhesivo):** Se debe utilizar cuando el contexto visual es esencial para la comprensión del texto que lo acompaña. Por ejemplo, mantener una pieza de hardware visible mientras el texto describe sus componentes internos.[2] Esto reduce la carga cognitiva al no obligar al usuario a recordar qué estaba viendo párrafos atrás.
- **Parallax (Paralaje):** Es ideal para crear una sensación de profundidad y atmósfera. Sin embargo, el paralaje excesivo es un factor de riesgo para el mareo o trastornos vestibulares. La ingeniería debe limitar la disparidad de velocidades entre capas a una

relación máxima de 1:2 para evitar la desorientación.[19]

3. Ingeniería Frontend y Rendimiento Crítico

Para un desarrollador Senior, el rendimiento no es un "extra", es la narrativa misma. Un scrollytelling que pierde frames rompe la suspensión de la incredulidad y frustra al usuario. La clave está en comprender el pipeline de renderizado del navegador y minimizar el trabajo del hilo principal.[21]

CSS Scroll-driven Animations API (W3C)

La nueva API de CSS permite declarar animaciones de scroll que el navegador puede optimizar de forma autónoma. Esto es fundamental para reemplazar librerías de JavaScript en tareas simples.[4]

- **Scroll Progress Timelines:** Permiten que una animación progrese de 0% a 100% basándose en el desplazamiento total de un contenedor.[7]
- **View Progress Timelines:** Permiten que una animación se active en función de la visibilidad de un elemento dentro del viewport (similar a IntersectionObserver pero sin la latencia de JS).[4]

Propiedades críticas de esta API:

- animation-timeline: Vincula la animación a un cronograma de scroll definido.[23]
- scroll(): Función para definir el scroller (nearest, root, self) y el eje (block, inline).[7]
- animation-range: Define los puntos exactos de inicio y fin de la animación dentro del scroll (ej: entry 0% exit 100%).[4]

Optimización de GPU y Prevención de Layout Thrashing

El "Layout Thrashing" ocurre cuando el código lee y escribe propiedades del DOM que fuerzan al navegador a recalcular el diseño de la página repetidamente durante el scroll. Para evitarlo, un ingeniero Senior debe seguir estas reglas estrictas [22]:

- **Propiedades Seguras (Composite Only):** Solo se deben animar transform (translate, scale, rotate) y opacity. Estas propiedades no requieren recalcular el layout ni el pintado (paint), operando directamente en la GPU.[22]
- **Evitar Propiedades de Layout:** Nunca animar top, left, width, height, margin o padding. Estas propiedades son "caras" computacionalmente y garantizan jank en dispositivos móviles.[22]
- **Uso de will-change:** Se debe aplicar will-change: transform, opacity; solo a los elementos que realmente se van a mover, para que el navegador los asigne a una capa de GPU independiente. No se debe abusar de esta propiedad, ya que consume memoria de video valiosa.[3]

JavaScript Moderno: GSAP y ScrollTrigger en Next.js

GSAP sigue siendo la herramienta más potente para secuencias complejas. En entornos de React/Next.js 14/15, la implementación debe ser "limpia" para evitar fugas de memoria y comportamientos erráticos en el modo estricto de React.[24]

Reglas de Oro (Golden Rules) para GSAP en React:

1. **Uso de useGSAP():** Este hook oficial reemplaza a useEffect o useLayoutEffect, gestionando automáticamente la creación y destrucción de contextos de animación.[3]
2. **Limpieza de Contextos:** Es imperativo que todas las animaciones se creen dentro de un contexto de GSAP que se pueda revertir (ctx.revert()) al desmontar el componente. Esto previene que los ScrollTriggers queden activos en memoria o colisionen con nuevas instancias.[24]
3. **Registro de Plugins:** Plugins como ScrollTrigger deben registrarse globalmente una sola vez para evitar sobrecarga.[3]
4. **Refresco de Posiciones:** En aplicaciones SPA donde el contenido puede cambiar dinámicamente, se debe invocar ScrollTrigger.refresh() después de que los elementos hayan terminado de renderizarse para asegurar que los puntos de activación sean correctos.[3]

Framer Motion: Patrones para el App Router de Next.js

Framer Motion es excelente por su simplicidad declarativa, pero presenta desafíos en Next.js debido a su dependencia de APIs del cliente.[26]

- **Directiva "use client":** Todos los componentes que utilicen motion o hooks de scroll deben ser Client Components, ya que Framer Motion necesita acceder al objeto window para rastrear el scroll.[27]
- **useScroll y useTransform:** Para un rendimiento óptimo, se deben usar MotionValues devueltos por useScroll. Estos valores se actualizan fuera del ciclo de renderizado de React, evitando que el componente se re-renderice en cada píxel de scroll.[6]
- **Intercalado de Componentes:** Un patrón eficiente es tener un contenedor de servidor que traiga los datos y un pequeño componente de cliente que aplique la animación. Esto maximiza el SEO y minimiza el bundle de JS enviado.[30]

Smooth Scrolling con Lenis

El "Smooth Scrolling" mejora la percepción de calidad, pero si se hace mal, destruye la accesibilidad. Lenis se ha convertido en el estándar por ser "respetuoso" con el scroll nativo.[32]

- **Configuración Óptima:** Se recomienda un valor de lerp (interpolación lineal) entre 0.05 y 0.1 para un balance entre suavidad y respuesta táctil.[34]
- **Manejo de Accesibilidad:** Lenis permite detectar la preferencia de movimiento reducido

del usuario y desactivar la suavidad automáticamente, algo crítico para evitar el mareo.[36]
- **Compatibilidad con Sticky:** A diferencia de librerías antiguas (como Locomotive Scroll), Lenis no utiliza transformaciones de contenedor para simular el scroll, lo que permite que position: sticky nativo funcione sin problemas.[35]

4. Técnicas Visuales Avanzadas (Arte Técnico)

El nivel "Senior" se demuestra en la capacidad de manipular medios pesados con la ligereza de un elemento vectorial.

Video Scrubbing: Fluidez Cuadro por Cuadro

Vincular el progreso de un video al scroll es una de las tareas más demandantes. Existen tres enfoques principales con diferentes compensaciones de rendimiento.[11]

Método	Técnica	Ventajas	Desventajas
Video Tag (currentTime)	Manipulación directa del tiempo de reproducción.	Simple de implementar.	Los videos altamente comprimidos (con GOP largo) no pueden retroceder o avanzar fluidamente sin saltos.
Image Sequence (Canvas)	Dibujar cientos de JPEGs o WebPs en un canvas secuencialmente.	Máxima fluidez y control total.	Impacto masivo en el ancho de banda y carga inicial.
WebCodecs API	Decodificación manual de frames en el navegador.	Balance perfecto: archivos pequeños, fluidez de canvas.	Complejidad técnica extrema; requiere gestión manual de memoria GPU. [11]

Estrategia de Producción para Video Scrubbing: Para 2025, el uso de WebCodecs es la recomendación técnica para experiencias "premium". Permite decodificar frames individuales de un archivo de video estándar y pintarlos en un canvas. Un punto crítico es la gestión de memoria: cada objeto VideoFrame debe cerrarse manualmente (frame.close()) para evitar que

la memoria de la GPU se agote rápidamente durante el desplazamiento rápido.[11]

Masking & Reveal: El Efecto de Revelación Premium

Las técnicas de enmascaramiento permiten transiciones visuales impactantes, como textos que emergen de "oro líquido" o imágenes que se revelan a través de formas orgánicas.[37]

- **clip-path Dinámico:** Utilizar polígonos o insets para revelar contenido. Es extremadamente eficiente ya que el navegador trata el clip como una operación geométrica básica en la GPU.[38]
- **mask-image con Gradientes:** Permite transiciones suaves y desvanecimientos imposibles con opacidad estándar. Se puede animar la posición del gradiente mediante variables CSS vinculadas al scroll.[39]
- **Técnicas de Capas (Pinning & Layering):** El uso de secciones con position: sticky permite crear efectos de "revelación por apilamiento" donde una sección nueva parece deslizarse sobre la anterior, manteniendo el contenido de fondo fijo mientras se transforma.[17]

5. Accesibilidad (A11Y): El Punto Crítico

Un sitio web que causa mareos o es inavegable no es una obra maestra técnica, es un fallo de ingeniería. La accesibilidad en scrollytelling se centra en el consentimiento del usuario y la preservación de los mecanismos de navegación estándar.[20]

La Media Query prefers-reduced-motion

Es imperativo honrar la configuración del sistema del usuario. Muchos usuarios sufren de sensibilidad al movimiento, donde las animaciones de scroll pueden disparar respuestas vestibulares (náuseas, vértigo).[20]

- **Implementación Ética:** Al detectar prefers-reduced-motion: reduce, el desarrollador debe desactivar todas las animaciones de paralaje y transformaciones agresivas, reemplazándolas por cambios instantáneos o fades de opacidad sutiles.[20]
- **Automatización:** Librerías como Smooth Scroll y wrappers de Lenis para React ofrecen ganchos nativos para manejar esto sin intervención manual extensa.[20]

Navegación por Teclado y Screen Readers

El scrollytelling a menudo oculta elementos o los mueve fuera del flujo visual, lo que puede romper la navegación con la tecla Tab.

1. **Gestión de Foco:** Si una animación desplaza al usuario visualmente, el foco del teclado debe seguir ese movimiento. Al usar botones de "scroll-to-top" o navegación por anclajes, el foco debe moverse físicamente al nuevo contenedor para que los usuarios de lectores de pantalla no se pierdan en el "espacio muerto" del DOM.[20]

2. **Visibilidad Semántica:** Los elementos que están ocultos mediante opacidad 0 o clip-path todavía son detectables por los lectores de pantalla a menos que se use aria-hidden="true" o display: none. Un ingeniero Senior debe asegurar que el orden de lectura del DOM coincida con el orden de la narrativa visual.[20]

El Debate del Scroll Hijacking

El "secuestro del scroll" (scrolljacking) es la alteración forzada del comportamiento de desplazamiento nativo, como bloquear al usuario en una sección hasta que termine una animación.[18]

- **Por qué es una mala práctica:** Frustra las expectativas del usuario, rompe la consistencia externa del sistema operativo y a menudo impide que los usuarios con dispositivos de entrada no convencionales puedan navegar.[20]
- **La Alternativa Respetuosa:** En lugar de bloquear el scroll, se deben usar animaciones vinculadas a la posición ("Scrubbed animations"). Esto permite que el usuario decida la velocidad de la historia y pueda retroceder o saltar secciones libremente, manteniendo el control total de su dispositivo.[18]

6. Sección de Arquitectura: Comparativa de Implementación Modular

Para un proyecto de Next.js 14/15 de alto rendimiento, la arquitectura sugerida es un modelo híbrido que maximiza la eficiencia.

Componente	Tecnología Recomendada	Justificación Técnica
Contenedor Global de Scroll	Lenis (React Lenis)	Proporciona una base de suavidad sin romper el scroll nativo y con soporte de accesibilidad integrado.[33]
Parallax Simple & Reading Progress	CSS Scroll-driven API	Ejecución fuera del hilo principal para máxima fluidez en navegadores modernos.[4]
Secuencias Narrativas Complejas	GSAP + useGSAP	Control absoluto sobre tiempos, callbacks y orquestación de múltiples

		elementos.[3]
Micro-interacciones y UI Animada	Framer Motion	Facilidad para animar estados de componentes React y manejo de MotionValues eficientes.[6]
Visualización de Producto (360°/Video)	Canvas + WebCodecs	Evita la latencia del tag <video> y el peso de las secuencias de imágenes masivas.[11]

7. Sección de Patrones de Código (Next.js 14/15)

A continuación, se presentan patrones de código optimizados para su implementación inmediata en entornos modernos.

Patrón 1: Video Scrubbing de Alto Rendimiento (Canvas + RequestAnimationFrame)

Este patrón evita el uso de currentTime y utiliza un sistema de dibujado en canvas para garantizar fluidez absoluta.

JavaScript

```javascript
"use client";
import { useRef, useEffect } from 'react';
import { useScroll, useTransform, useMotionValueEvent } from 'framer-motion';

export const VideoScrubCanvas = ({ totalFrames, framePaths }) => {
  const canvasRef = useRef(null);
  const imagesRef = useRef();
  const { scrollYProgress } = useScroll();

  // Mapear el scroll a los índices de los frames
  const frameIndex = useTransform(scrollYProgress, , [0, totalFrames - 1]);

  useEffect(() => {
    // Pre-carga de imágenes para evitar parpadeos
    framePaths.forEach((path, i) => {
```

```
    const img = new Image();
    img.src = path;
    imagesRef.current[i] = img;
  });
}, [framePaths]);

const renderFrame = (index) => {
  const ctx = canvasRef.current?.getContext('2d');
  const img = imagesRef.current[Math.round(index)];
  if (ctx && img) {
    ctx.clearRect(0, 0, canvasRef.current.width, canvasRef.current.height);
    ctx.drawImage(img, 0, 0);
  }
};

useMotionValueEvent(frameIndex, "change", (latest) => {
  requestAnimationFrame(() => renderFrame(latest));
});

return <canvas ref={canvasRef} width={1920} height={1080} className="sticky top-0 w-full h-screen
object-cover" />;
};
```

Patrón 2: Sticky Reveal con GSAP y Next.js 14

Implementación del patrón donde una imagen queda fija mientras el texto pasa, utilizando el hook useGSAP para una limpieza segura de memoria.[3]

JavaScript

```
"use client";
import { useRef } from 'react';
import gsap from 'gsap';
import { ScrollTrigger } from 'gsap/ScrollTrigger';
import { useGSAP } from '@gsap/react';

gsap.registerPlugin(ScrollTrigger);

export default function StickyRevealSection() {
  const containerRef = useRef(null);
```

```
  const imageRef = useRef(null);

  useGSAP(() => {
    // Pinning de la imagen central
    ScrollTrigger.create({
      trigger: containerRef.current,
      start: "top top",
      end: "bottom bottom",
      pin: imageRef.current,
      pinSpacing: false,
    });

    // Animación de capas de texto que pasan por encima
    gsap.from(".text-layer", {
      scrollTrigger: {
        trigger: ".text-layer",
        start: "top bottom",
        end: "top center",
        scrub: true,
      },
      opacity: 0,
      y: 100,
    });
  }, { scope: containerRef }); // Scope asegura que solo se animen elementos internos

  return (
    <div ref={containerRef} className="relative min-h-[300vh]">
      <div ref={imageRef} className="h-screen w-full bg-black flex items-center justify-center">
        <img src="/product-hero.webp" className="w-1/2" alt="Product" />
      </div>
      <div className="absolute top-0 left-0 w-full">
        <section className="text-layer h-screen flex items-center justify-end p-20">
          <h2 className="text-white text-6xl">Innovación radical.</h2>
        </section>
        <section className="text-layer h-screen flex items-center justify-end p-20">
          <h2 className="text-white text-6xl">Rendimiento sin límites.</h2>
        </section>
      </div>
    </div>
  );
}
```

8. Sección de Checklist de Calidad: Performance y

Accesibilidad

Un desarrollo de scrollytelling solo se considera terminado cuando supera los siguientes umbrales de calidad.[21]

Auditoría de Rendimiento (Performance Metrics)

Métrica	Umbral Senior	Herramienta de Medición
FPS (Frames Per Second)	60 FPS constantes (especialmente durante scrubbing).	Chrome DevTools (FPS Meter).
TBT (Total Blocking Time)	< 200 ms.	Lighthouse / WebPageTest. [21]
CLS (Cumulative Layout Shift)	< 0.1.	Search Console / Lighthouse. [21]
LCP (Largest Contentful Paint)	< 2.5 s.	Core Web Vitals. [21]
Uso de Memoria GPU	Sin fugas incrementales durante el desplazamiento continuo.	Chrome Task Manager (GPU Memory).

Auditoría de Accesibilidad y UX

- [ ] **Reduced Motion:** ¿El sitio desactiva el paralaje y los movimientos bruscos si el usuario lo solicita en su sistema? [20]
- [ ] **Keyboard Focus:** ¿Es posible navegar por toda la historia usando solo la tecla Tab sin quedar atrapado en bucles de foco? [20]
- [ ] **Alt Text & Aria:** ¿Los elementos visuales clave tienen descripciones alternativas y los estados animados se comunican a los lectores de pantalla? [42]
- [ ] **Scroll Hijacking Check:** ¿El usuario siente que tiene el control de la velocidad? ¿Puede usar la barra de desplazamiento nativa sin resistencia artificial? [32]
- [ ] **Mobile Optimization:** ¿El scrollytelling funciona con gestos de "thumb-swipe" sin causar saltos accidentales de sección? [45]
- [ ] **Fallback de Video:** ¿Existe una imagen estática de alta calidad si el video no carga o el navegador no soporta el formato? [11]

La excelencia en el scrollytelling no reside en la complejidad de las animaciones, sino en su sutileza y en la solidez de la ingeniería que las sustenta. Un manual técnico como este sirve como guía para asegurar que la innovación visual nunca comprometa la usabilidad y que el rendimiento sea el lenguaje silencioso de una experiencia de usuario superior.[16]

Obras citadas

1. scrollytelling - CX Fusion, fecha de acceso: febrero 6, 2026, https://cxfusion.io/en/scrollytelling/
2. Shaga Odyssey Website Design Analysis | DesignRush, fecha de acceso: febrero 6, 2026, https://www.designrush.com/best-designs/websites/shaga-odyssey-website-design
3. Setting Up GSAP with Next.js: 2025 Edition | by Thomas Augot ..., fecha de acceso: febrero 6, 2026, https://javascript.plainenglish.io/setting-up-gsap-with-next-js-2025-edition-bcb86e48eab6
4. CSS scroll-driven animations - CSS | MDN, fecha de acceso: febrero 6, 2026, https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Scroll-driven_animations
5. The Great Debate: CSS vs JavaScript in Modern Web Development | by Jon Christie, fecha de acceso: febrero 6, 2026, https://jcircle9.medium.com/the-great-debate-css-vs-javascript-in-modern-web-development-15deb5b7f605
6. useScroll — React scroll-linked animations - Motion.dev, fecha de acceso: febrero 6, 2026, https://motion.dev/docs/react-use-scroll
7. Scroll-driven Animations Module Level 1, fecha de acceso: febrero 6, 2026, https://drafts.csswg.org/scroll-animations-1/
8. A guide to Scroll-driven Animations with just CSS | WebKit, fecha de acceso: febrero 6, 2026, https://webkit.org/blog/17101/a-guide-to-scroll-driven-animations-with-just-css/
9. UX/UI Jobs in Singapore 2024 - NTUC LearningHub, fecha de acceso: febrero 6, 2026, https://www.ntuclearninghub.com/-/blogs/ux-ui-jobs-singapore
10. Using useGSAP to create a ScrollTrigger scene in Next.js - Tech via Foci, fecha de acceso: febrero 6, 2026, https://tech.viafoci.com/using-usegsap-to-create-a-scrolltrigger-scene-in-next-js/
11. A Tutorial: WebCodecs Video Scroll Synchronization | by Keng Lim ..., fecha de acceso: febrero 6, 2026, https://lionkeng.medium.com/a-tutorial-webcodecs-video-scroll-synchronization-8b251e1a1708
12. Designing for the 8-Second Mind: Cognitive UX Strategy in the Age of Digital Overload, fecha de acceso: febrero 6, 2026, https://medium.com/@Atrams/designing-for-the-8-second-mind-cognitive-ux-st

rategy-in-the-age-of-digital-overload-99fecd1db7a2
13. Conversion Rate Optimization Research & Best Practices - Jackal, fecha de acceso: febrero 6, 2026, https://www.tryjackal.com/research
14. The Impact of Scrollytelling on the Reading Experience of Long-Form Journalism, fecha de acceso: febrero 6, 2026, https://www.researchgate.net/publication/374109425_The_Impact_of_Scrollytelling_on_the_Reading_Experience_of_Long-Form_Journalism
15. The Impact of Scrollytelling on the Reading Experience of Long-Form Journalism - Digital Economy Wales, fecha de acceso: febrero 6, 2026, https://digitaleconomy.wales/documents/ecce-2023-papers/06-The-Impact-of-Scrollytelling.pdf
16. Designing the Future: Top UX Trends Shaping 2025 and Beyond - Modern Opulent Gazette, fecha de acceso: febrero 6, 2026, https://www.modernopulentgazette.com/post/designing-the-future-top-ux-trends-shaping-2025-and-beyond
17. Introducing “scrub on scroll” for video, slideshows, Lottie and more - Maglr, fecha de acceso: febrero 6, 2026, https://www.maglr.com/updates/introducing-scrub-on-scroll-for-video-slideshows-lottie-and-more
18. scroll-experience | Skills.lc - AI Agent Skills, fecha de acceso: febrero 6, 2026, https://skills.lc/NeverSight/skills_feed/neversight-skills-feed-data-skills-md-poletron-custom-rules-scroll-experience-skill-md
19. 30 Days Training Guide | PDF | Search Engine Optimization - Scribd, fecha de acceso: febrero 6, 2026, https://www.scribd.com/document/951476279/30-Days-Training-Guide
20. Paint the Picture, Not the Frame: How Browsers Provide Everything ..., fecha de acceso: febrero 6, 2026, https://alistapart.com/article/paint-the-picture-not-the-frame/
21. Front-End Performance Checklist 2021 (PDF, Apple Pages, MS Word) - Smashing Magazine, fecha de acceso: febrero 6, 2026, https://www.smashingmagazine.com/2021/01/front-end-performance-2021-free-pdf-checklist/
22. GSAP Setup for Next.js 13/14, fecha de acceso: febrero 6, 2026, https://gsap.com/community/forums/topic/39864-gsap-setup-for-nextjs-1314/
23. scroll() - CSS - MDN Web Docs, fecha de acceso: febrero 6, 2026, https://developer.mozilla.org/en-US/docs/Web/CSS/Reference/Properties/animation-timeline/scroll
24. GSAP is not working properly after refreshing the page (in react.js) - Stack Overflow, fecha de acceso: febrero 6, 2026, https://stackoverflow.com/questions/75813478/gsap-is-not-working-properly-after-refreshing-the-page-in-react-js
25. gsap timeline in react - Stack Overflow, fecha de acceso: febrero 6, 2026, https://stackoverflow.com/questions/71011441/gsap-timeline-in-react
26. Getting Started: Server and Client Components - Next.js, fecha de acceso: febrero 6, 2026,

https://nextjs.org/docs/app/getting-started/server-and-client-components
27. Resolving Framer Motion Compatibility in Next.js 14: The 'use client' Workaround - Medium, fecha de acceso: febrero 6, 2026, https://medium.com/@dolce-emmy/resolving-framer-motion-compatibility-in-next-js-14-the-use-client-workaround-1ec82e5a0c75
28. Framer Motion and Next JS server side rendering : r/nextjs - Reddit, fecha de acceso: febrero 6, 2026, https://www.reddit.com/r/nextjs/comments/13bxfho/framer_motion_and_next_js_server_side_rendering/
29. Framer motion nextjs useScroll not working when attached to an element - Stack Overflow, fecha de acceso: febrero 6, 2026, https://stackoverflow.com/questions/77125691/framer-motion-nextjs-usescroll-not-working-when-attached-to-an-element
30. Client vs Server Components in Next.js: What Goes Where? | by Saroj Bist - Medium, fecha de acceso: febrero 6, 2026, https://medium.com/@Saroj_bist/client-vs-server-components-in-next-js-what-goes-where-74badf8c5620
31. Server Components vs. Client Components in Next.js: Differences, Pros, and Cons - DEV Community, fecha de acceso: febrero 6, 2026, https://dev.to/oskarinmix/server-components-vs-client-components-in-nextjs-differences-pros-and-cons-389f
32. Issue with SmoothScroll feature from Lenis Scroll - WordPress.org, fecha de acceso: febrero 6, 2026, https://wordpress.org/support/topic/issue-with-smoothscroll-feature-from-lenis-scroll/
33. Get buttery smooth scrolling in your Next.js app - Gaurav Shukla, fecha de acceso: febrero 6, 2026, https://www.gshukla.in/articles/smooth-scrolling-in-nextjs-with-lenis
34. How to implement smooth scrolling in Next.js with Lenis and GSAP - DevDreaming, fecha de acceso: febrero 6, 2026, https://devdreaming.com/blogs/nextjs-smooth-scrolling-with-lenis-gsap
35. Lenis Smooth Scroll: Step-by-Step Integration Guide in Webflow - Digidop, fecha de acceso: febrero 6, 2026, https://www.digidop.com/blog/lenis-smooth-scroll
36. How to implement Lenis in Next.js - Bridger Tower, fecha de acceso: febrero 6, 2026, https://bridger.to/lenis-nextjs
37. Advanced CSS Clip-path and Masking Tricks for Modern Visual Effects - Medium, fecha de acceso: febrero 6, 2026, https://medium.com/uxdworld/advanced-css-clip-path-and-masking-tricks-for-modern-visual-effects-8c4a3715f10d
38. Clipping and Masking in CSS - CSS-Tricks, fecha de acceso: febrero 6, 2026, https://css-tricks.com/clipping-masking-css/
39. Apply effects to images with the CSS mask-image property | Articles - web.dev, fecha de acceso: febrero 6, 2026, https://web.dev/articles/css-masking
40. CSS Masking Module Level 1 - W3C, fecha de acceso: febrero 6, 2026, https://www.w3.org/TR/css-masking-1/

41. Jpeg Sequence (video) Scrub effect on scroll, but sticky centered - Framer Community, fecha de acceso: febrero 6, 2026, https://www.framer.community/c/developers/jpeg-sequence-video-scrub-effect-on-scroll-but-sticky-centered
42. The Future of Interactive Website Design: Trends Shaping Digital Experiences in 2026, fecha de acceso: febrero 6, 2026, https://onewebcare.com/blog/interactive-website-design/
43. Elegant scrolling in Next.JS - Medium, fecha de acceso: febrero 6, 2026, https://medium.com/@iamyuvraj456/elegant-scrolling-in-next-js-2d7f3fb4c8e1
44. How to Improve Website Engagement Metrics | Webolutions, fecha de acceso: febrero 6, 2026, https://webolutionsmarketingagency.com/blog/performance-analytics/how-to-improve-website-engagement-metrics/
45. Mobile vs. Desktop Popup Behavior | 2025 Conversion Data - Popupsmart, fecha de acceso: febrero 6, 2026, https://popupsmart.com/blog/mobile-vs-desktop-popup-behavior